

SIMATS ENGINEERING

ASSESSMENT TOOL 2

Annexure A – Pseudocode Writing Task (Algorithm Design)

Course Name	Computer Networks
Course Code	CSA07
Course Outcome	CO5
Description	Measure network performance using key metrics with simulation tools. (BL4, B
Assessment Weightage	30%

Submitted by

Name: S. KARTHIKEYAN

Register No: 192419048

Code of Conduct

I, S. Karthikeyan (Reg. No: 192419048), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Question 1: Throughput & Packet Loss Calculator

Scenario: A student records total data received as 8000 bits in 4 seconds, with 200 packets sent and 180 packets received in a Cisco Packet Tracer simulation. Write a pseudocode algorithm to calculate throughput (bps) and packet loss (%), validate against zero time/packets, classify the network as Efficient/Inefficient, and print a full summary.

Problem Understanding

Inputs: total_bits_received, time_seconds, packets_sent, packets_received. **Outputs:** throughput (bps), packet_loss (%), network classification, and a summary report. **Constraints:** time_seconds must not be 0 (division by zero in throughput), and packets_sent must not be 0 (division by zero in packet-loss percentage).

Pseudocode

```
BEGIN ThroughputAndPacketLoss

// ---- Inputs ----
DECLARE total_bits_received, time_seconds AS REAL
DECLARE packets_sent, packets_received AS INTEGER
DECLARE throughput, packet_loss_percent AS REAL
DECLARE network_status AS STRING

total_bits_received <- 8000
time_seconds      <- 4
packets_sent      <- 200
packets_received  <- 180

// ---- Validation ----
IF time_seconds = 0 THEN
    DISPLAY "Error: Time cannot be zero. Cannot compute throughput."
    STOP
END IF

IF packets_sent = 0 THEN
    DISPLAY "Error: Packets sent cannot be zero. Cannot compute packet loss."
    STOP
END IF

// ---- Core Calculations ----
throughput <- total_bits_received / time_seconds      // bits per second
packet_loss_percent <- ((packets_sent - packets_received) / packets_sent) * 100

// ---- Classification ----
IF throughput > 1500 AND packet_loss_percent < 10 THEN
    network_status <- "Efficient"
ELSE
    network_status <- "Inefficient"
END IF

// ---- Output Summary ----
DISPLAY "----- NETWORK PERFORMANCE SUMMARY -----"
DISPLAY "Throughput          : ", throughput, " bps"
DISPLAY "Packet Loss         : ", packet_loss_percent, " %"
DISPLAY "Packets Sent          : ", packets_sent
```

```
DISPLAY "Packets Received      : ", packets_received
DISPLAY "Network Status       : ", network_status
DISPLAY "-----"
```

```
END ThroughputAndPacketLoss
```

Trace / Verification with Given Values

throughput = $8000 / 4 = 2000$ bps (> 1500 , condition 1 satisfied). packet_loss = $((200 - 180) / 200) \times 100 = 10\%$ (NOT $< 10\%$, so condition 2 fails). Since both conditions must hold for 'Efficient', the algorithm correctly classifies this network as **Inefficient** despite the healthy throughput — illustrating exactly why the AND condition (not OR) is the correct logical design choice here.

Question 2: Bandwidth Utilization Monitor

Scenario: An organization has multiple network links connecting its branches. Write a pseudocode algorithm that periodically collects the bandwidth usage of each link, stores values in an array/list, continuously monitors all links in a loop, and recommends switching to a backup link when utilization exceeds 80%.

Problem Understanding

Inputs: number_of_links, per-link bandwidth utilization readings (collected periodically). **Outputs:** a running utilization log per link, and an overload alert/recommendation whenever a link exceeds the 80% threshold. **Constraints:** monitoring must run continuously (indefinitely, at fixed intervals) rather than as a one-time check, and each link must be checked independently every cycle.

Pseudocode

```
BEGIN BandwidthUtilizationMonitor

    DECLARE num_links AS INTEGER
    DECLARE link_names[num_links] AS STRING
    DECLARE utilization[num_links] AS REAL
    DECLARE backup_link[num_links] AS STRING
    DECLARE monitor_interval AS INTEGER    // seconds between checks
    DECLARE THRESHOLD AS REAL <- 80

    // ---- Setup: link identities and paired backups ----
    INITIALIZE link_names, backup_link WITH configured branch-link data
    monitor_interval <- 30

    // ---- Continuous Monitoring Loop ----
    WHILE TRUE DO

        FOR i <- 1 TO num_links DO

            utilization[i] <- READ_CURRENT_BANDWIDTH_USAGE(link_names[i])

            DISPLAY "Link: ", link_names[i], " | Utilization: ", utilization[i], "%"

            IF utilization[i] > THRESHOLD THEN
                DISPLAY "ALERT: ", link_names[i], " is overloaded (",
                    utilization[i], "%)."
                DISPLAY "Recommendation: Switch traffic to backup link -> ",
                    backup_link[i]
            ELSE
                DISPLAY "Status: ", link_names[i], " operating normally."
            END IF

        END FOR

        WAIT(monitor_interval)    // pause before next monitoring cycle

    END WHILE

END BandwidthUtilizationMonitor
```

How This Supports Efficient Bandwidth Management

By continuously looping over every link (rather than checking once), the algorithm catches overload conditions as soon as they occur instead of after a delay or complaint. The 80% threshold acts as an early-warning buffer – well before a link would hit 100% and start dropping packets, traffic can already be redirected to its paired backup link. This prevents congestion from building up on any single path, keeps latency predictable across the network, and gives the administrator a clear, per-link audit trail (via the DISPLAY log) to plan capacity upgrades where overload alerts recur most often.

Question 3: Adaptive Best-Path Selection Across Branch Links

Scenario: Six branch offices are connected through multiple links with varying bandwidth, delay, and utilization. Write a pseudocode algorithm that periodically collects these metrics, computes a link-quality score, selects the best path per branch, detects overload/failure, auto-switches to an alternate path, and alerts the administrator.

Problem Understanding

Inputs: for every link to every branch – bandwidth, delay, and utilization, refreshed at regular intervals. **Outputs:** a computed quality score per link, the selected best path per branch, and failure/overload alerts with automatic failover. **Constraints:** the scoring must combine all three metrics (not distance alone), monitoring must repeat on a loop, and a failed/overloaded link must never remain the active path once detected.

Pseudocode

```
BEGIN BestPathSelection

    DECLARE num_branches AS INTEGER <- 6
    DECLARE links_per_branch[num_branches] AS INTEGER
    DECLARE bandwidth[num_branches][], delay[num_branches][],
        utilization[num_branches][] AS REAL
    DECLARE quality_score[num_branches][] AS REAL
    DECLARE active_path[num_branches] AS INTEGER
    DECLARE OVERLOAD_LIMIT AS REAL <- 80
    DECLARE monitor_interval AS INTEGER <- 30

    // Weights for the composite quality score (tunable)
    DECLARE W_BW AS REAL <- 0.5
    DECLARE W_DELAY AS REAL <- 0.3
    DECLARE W_UTIL AS REAL <- 0.2

    WHILE TRUE DO

        FOR branch <- 1 TO num_branches DO

            best_score <- -INFINITY
            best_link <- -1

            FOR link <- 1 TO links_per_branch[branch] DO

                bandwidth[branch][link] <- READ_BANDWIDTH(branch, link)
                delay[branch][link] <- READ_DELAY(branch, link)
                utilization[branch][link] <- READ_UTILIZATION(branch, link)

                // ---- Composite Link Quality Score ----
                // higher bandwidth is good; higher delay/utilization is bad
                quality_score[branch][link] <-
                    (W_BW * bandwidth[branch][link]) -
                    (W_DELAY * delay[branch][link]) -
                    (W_UTIL * utilization[branch][link])

                // ---- Failure / Overload Detection ----
                IF utilization[branch][link] >= 100 OR
                    LINK_STATUS(branch, link) = "DOWN" THEN
```

```

        DISPLAY "ALERT: Link ", link, " to Branch ", branch,
            " has FAILED or is saturated."
        quality_score[branch][link] <- -INFINITY // exclude from selection

    ELSE IF utilization[branch][link] > OVERLOAD_LIMIT THEN
        DISPLAY "ALERT: Link ", link, " to Branch ", branch,
            " is overloaded (", utilization[branch][link], "%)."
    END IF

    // ---- Track the best-scoring viable link ----
    IF quality_score[branch][link] > best_score THEN
        best_score <- quality_score[branch][link]
        best_link <- link
    END IF

END FOR

// ---- Switch path if the best link changed ----
IF best_link <> active_path[branch] THEN
    DISPLAY "Switching Branch ", branch, " to Link ", best_link,
        " (Score: ", best_score, ")"
    active_path[branch] <- best_link
END IF

DISPLAY "Branch ", branch, " -> Active Path: Link ", active_path[branch]

END FOR

WAIT(monitor_interval)

END WHILE

END BestPathSelection

```

How This Improves Reliability, Routing Efficiency & Fault Tolerance

Reliability: because the -INFINITY penalty removes a failed or saturated link from consideration the moment it is detected, traffic is never left routed over a dead or overloaded path – failover is automatic rather than depending on a manual intervention. **Routing efficiency:** scoring on a weighted combination of bandwidth, delay, and utilization (instead of shortest distance alone) means the chosen path genuinely reflects current usable capacity, so traffic isn't sent down a technically 'shorter' link that is actually slow or congested. **Fault tolerance and adaptability:** the outer WHILE loop re-evaluates every branch's links at fixed intervals, so the active path for each branch continuously adapts to changing daytime traffic patterns rather than being fixed once at startup, and every switch and alert is logged for the administrator to review network health trends over time.